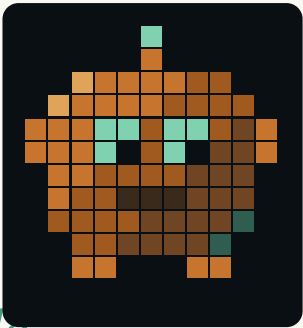


The editor your agent wishes it had.

A grep-fast, zero-dependency CLI
built by an AI agent
for the way agents actually read & edit code.



CU · THE CURSOR, ALIVE

tarn is a tiny terminal editor and a structural command-line toolkit for AI coding agents: orient in a repo, jump to a symbol's definition or its uses, read one function, and edit by unit-of-meaning — all deterministic, --json-chainable, with meaningful exit codes. Pure Rust, **no crate dependencies**. And on large-file counting it is at parity with — or faster than — ripgrep, the fastest search tool there is. Every number in this paper was measured, not estimated.

DEPENDENCIES

0

pure Rust std; mmap, threads & SIMD via core/libc — no crates

COMMANDS

25

navigate · read · edit · config · verify

VS RIPGREP

1.3×

faster on a single 380 MB file (~42 vs ~57 ms); parity on many files

TESTS

124

+ adversarial review on every feature; ASan-clean

A FIELD GUIDE TO A SEARCH-AND-EDIT TOOL THAT EARNS AN AGENT'S TRUST

Benchmarks: 10-core Apple Silicon, warm cache, median of 15–21 runs, counting the 99,197 lines containing function in a ~380 MB corpus of real source. Reproduce with `tarn find -c` vs `rg -c` on any large file.

What's inside

01	Why agents fight their tools.	3
02	The surface: one binary, 25 commands.	3
03	Speed: at parity with ripgrep, zero deps.	4
04	How — mmap, all cores, SIMD, no crates.	5
05	An honest negative result: the index.	6
06	Trust is the product.	7

HOW TO READ THIS

Every claim is cashable.

The performance figures are medians of real runs on a real corpus against real tools (ripgrep 14, ugrep 7.5). Where a thing didn't work, it's in here too — section 05 is a feature we built, measured, and deleted. The tool is on GitHub; `tarn help --json` emits its whole surface for an agent to learn in one call.

01

THE PROBLEM

Why agents fight their tools.

A coding agent drives tools built for humans: open a file, scroll, count lines, hope the line numbers didn't move since the last read. The friction is real and it compounds across a session.

CONTEXT BURNED

whole files

read into the prompt just to find one symbol. `grep` gives lines without structure; `cat` gives everything without shape.

LINE DRIFT

off-by-N

an edit keyed to a line number that moved between read and write clobbers the wrong line. No guard, no recovery.

NO MACHINE SURFACE

prose

human tools answer in prose an agent must re-parse. No stable JSON, no exit-code contract to branch on.

02

THE SURFACE

One binary, 25 commands.

Structural, surgical, and self-describing. `tarn help --json` returns the whole manifest — every command, usage, example, and exit code — so an agent learns the tool in a single call.

NAVIGATE (NO LSP)

tree • outline • defs • refs • peek • find

Go-to-definition, find-references, repo map, and `grep` that hands you the enclosing definition of every hit — from heuristics, no language server.

EDIT BY MEANING

replace • insert • delete • apply • patch • batch • rename

`--def <name>` edits a whole function. `--expect` refuses if the target moved. `git diff | tarn patch` applies a unified diff, relocating drifted hunks.

CONFIG & VERIFY

json • toml • yaml • check • diff

Get/set/delete by path, format-preserving (comments & key order kept). Hygiene gate and a fast line-numbered diff. Exit codes: 0/1/2/3.

THE CONTRACT

Deterministic, guarded, chainable.

Every edit preserves line endings and never reflows untouched lines; `--dry-run` previews; multi-file `apply/patch` are atomic with rollback; `--json` on read & edit commands lets one command feed the next. An agent can act without a defensive re-read.

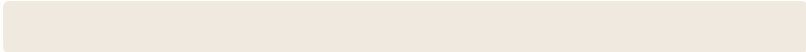
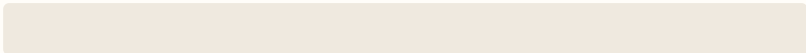
03 At parity with ripgrep. Zero dependencies.

THE SPEED

ripgrep is the fastest search tool in wide use — SIMD, mmap, a tuned parallel walk. tarn matches it on a directory and *beats* it on a single large file, with not one crate dependency.

COUNT MATCHES IN ONE 380 MB FILE — LOWER IS FASTER

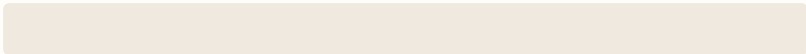
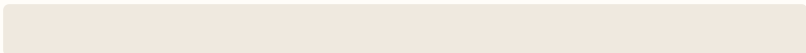
30,720 matching lines. Warm cache, median of 15 runs (figures bounce $\pm\sim 15\%$ run to run).

<code>tarn find -c</code>		~42 ms
<code>ripgrep -c</code>		~57 ms

tarn is $\sim 1.3\times$ faster than ripgrep here — and $\sim 45\times$ faster than the system grep (ugrep 7.5: $\approx 2.2\times$, off this chart).

COUNT MATCHES ACROSS 3,000 FILES (~ 380 MB) — LOWER IS FASTER

The recursive case. Warm cache, median of 15 runs.

<code>tarn find -c</code>		~52 ms
<code>ripgrep -c</code>		~48 ms

Parity — ripgrep edges it $\sim 1.1\times$. Both fan files out across cores; ripgrep's tuned per-file machinery still leads slightly.

THE HONEST CEILING

You can't count faster than you can read.

tarn counts a 380 MB file in ~ 42 ms; merely `cat`-ing it takes ~ 36 ms. tarn is at the memory-bandwidth wall — it counts in about the time it takes to read the bytes at all. So "10 $\times$ faster" on a single scan isn't an optimization, it's physics. (The 10 $\times$ hides elsewhere — see section 05.)

04 Fast with no crates.

THE HOW

ripgrep's raw byte-scan beats anything you'd hand-write in std. tarn wins the difference back three other ways — all standard library, no dependencies.

NO COPY

mmap (libc FFI)

Map the file and scan straight from the page cache — no fs::read copy of hundreds of MB. Same FFI tarn already uses for stty.

ALL CORES

std::thread

ripgrep searches one file on one thread; tarn splits it. \n is always a UTF-8 boundary, so each chunk counts independently and the sum is exact.

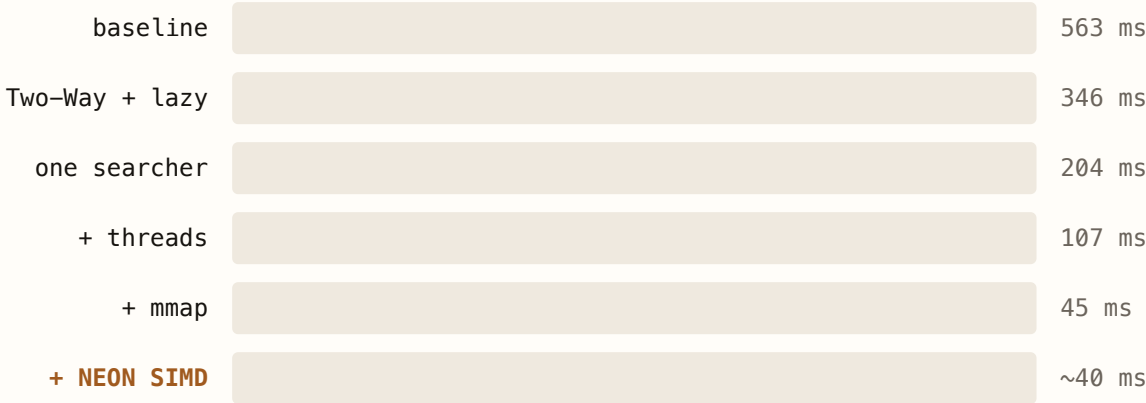
SIMD, STD

NEON via core::arch

A 16-byte-at-a-time memchr in NEON intrinsics — which live in core::arch, the standard library, not a crate. AddressSanitizer-clean.

THE JOURNEY — TARN FIND -C ON THE SAME 380 MB FILE

Each step is a std-only change. Lower is faster.



563 → ~40 ms: a ~14× speedup, entirely within the standard library.

BONUS: THE DIFF RENDERER

~7 s → ~26 ms on a 40k-line file.

A full-matrix LCS was a latent O(n·m) memory bomb. Trimming the common prefix/suffix and running the quadratic part only on the differing middle made a one-line change in a 40,000-line file diff in ~26 ms instead of ~7 s — backing every --diff flag. And tarn batch runs a whole command session in one process: ~10,500 edits/sec (~34× over per-call spawn), since an agent's edit loop is bottlenecked by OS process spawn, not by tarn.

05 Measure it. Then maybe delete it.

THE CARDINAL RULE

A tool earns trust by what it refuses to ship. We built a persistent navigation index to make repeated defs/refs sublinear — then measured it and pulled it.

THE PREMISE

91%

of a defs query on a 1,688-file repo is re-reading and re-parsing files that didn't change (~142 of ~156 ms). Cache that, the theory went, and queries go ~10×.

THE MEASUREMENT

1.4×

what the built, correct, never-state per-file cache actually delivered. Not 10×.

THE REASON

decode

each tarn call is a fresh process that re-reads and re-deserializes the whole index. The floor is decode + stat, not the parse we cached.

WHAT SHIPPED INSTEAD: NOTHING

1.4× doesn't justify persistent state.

A new on-disk format and a new failure surface, for 1.4×, on a tool whose whole appeal is "just point it at files" — not worth it. The prototype was reverted; the analysis is recorded in [docs/INDEX.md](#). A real 10× would need a resident daemon or an mmap'd zero-decode index — a much larger change, parked until the latency is actually felt. **The honest negative result is part of the spec.**

06 Trust is the product.

Speed gets a tool tried; trust gets it adopted. A search tool that returns one wrong-but-plausible answer gets pulled from the harness forever.

THE PRODUCT

Adversarial review on every feature. A ruthless reviewer agent gated each change before it shipped — and earned its keep: it caught a real `.env` data-loss bug (a non-UTF-8 byte made `set` wipe the file), a CRLF counting seam, and exit-code contract violations, each fixed at the root.

Counts proven exact. The SIMD scanner was differential-tested across 900+ cases against the previous implementation and against `rg/grep` — zero mismatches — and a 3,000-input fuzz vs a naive oracle. The unsafe NEON code is `AddressSanitizer-clean`.

Edits never corrupt. Line endings (LF/CRLF) and final-newline state are preserved on every edit; format-preserving config edits keep comments and key order; multi-file batches roll back as a unit.

124 tests, clippy & fmt clean, CI green. Every benchmark in this paper is reproducible from the repo. Where a number depended on the test machine, it's labelled. Nothing here is aspirational.

DEFINITION OF DONE

The tool an agent would choose.

Fast enough that you never reach for `grep`. Structural, so you jump to a symbol instead of scrolling. Guarded, so an edit can't silently land in the wrong place. Self-describing, so any agent learns it in one `tarn help --json`. And honest enough to delete the feature that didn't earn its place. Built by an agent — for the tools it wished it had.



tarn.

THE EDITOR YOUR AGENT WISHES IT HAD · BY AN AGENT, FOR AGENTS